```
        (
            [make] => VW
            [model] => Golf GTI
        )

    [1] => Array
        (
            [make] => VW
            [model] => Corrado VR6
        )

)
```

Our imaginary API call has returned a small data set of cars. Let's pretend that we're feeding this data into a template for rendering. Sadly, the template is expecting a different, associative structure, and we need to restructure the entire thing. Here, we have a function – `applySanity()` – which will do this for us. We use `array_walk()` to visit every element and transform it into a more sensible form.

The important thing to realize here is that the `$cars` array is changed in-place: `array_walk()` does not build a new array. We need to use the reference operator `&` in the signature of `applySanity()` otherwise we'll trigger PHP's copy on assignment.

If you're wondering if `array_walk()` can also manipulate keys, the answer is no. The callback function can have up to three arguments. `array_walk()` will first supply the iterated element (this argument is required), then two more optional parameters. The second callback argument is the key for the element, and the third callback argument is the third parameter passed to `array_walk()` (should you need to pass any other data into the callback function).

The full signature is `array_walk(array $array, callable $callback, mixed $anything): bool` and the signature of `$callback` will be `function (mixed $value, int|string $key, mixed $anything): void`. However, any attempt to access `$key` by reference will be ignored (and generate a warning error). Let's try to manipulate keys of the parent array as we re-structure:

```
$cars = [
    'car1' => [
        ['make', 'VW'], ['model', 'Golf GTI']
    ],
];
```